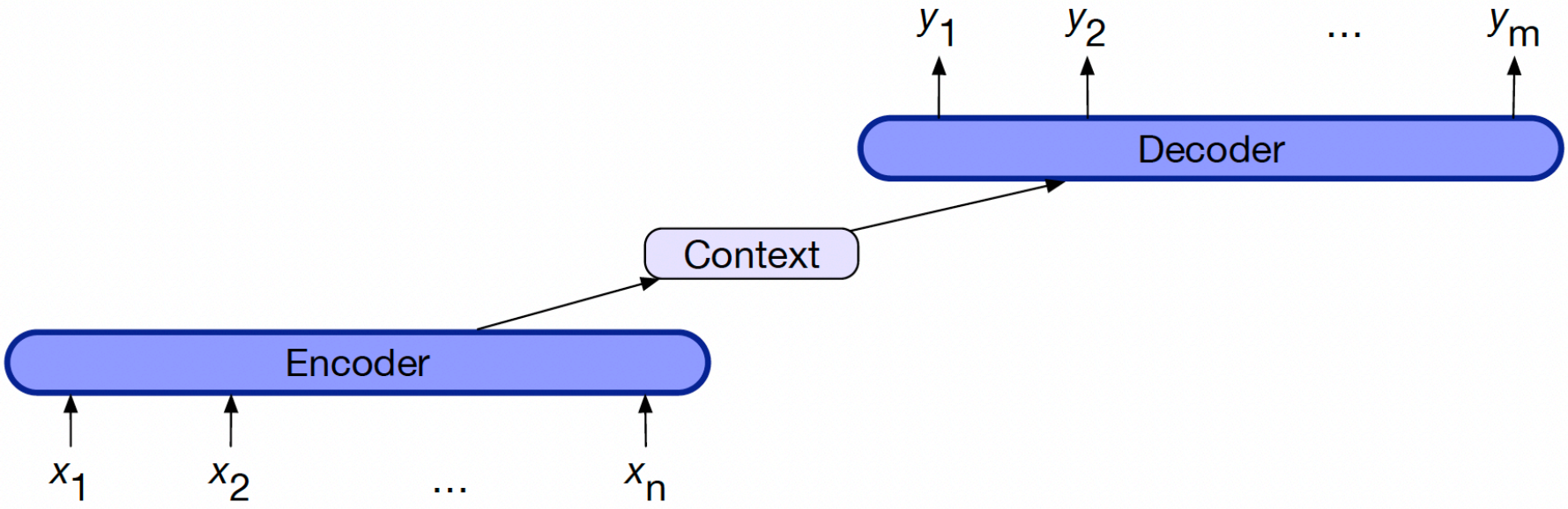
Neural Networks For NLP

Encoder- Decoder

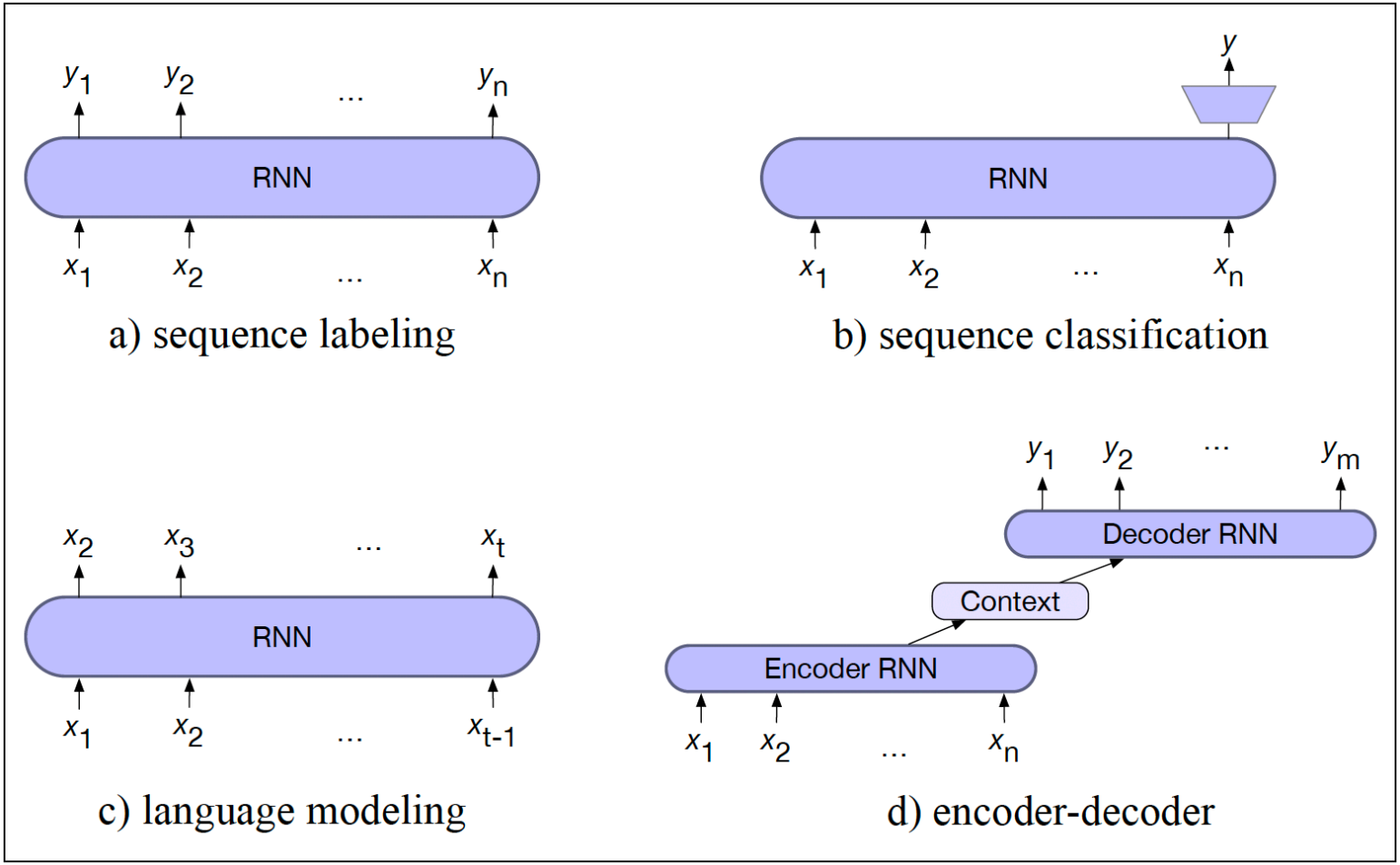
3 components of an encoder-decoder

1. An encoder that accepts an input sequence, $x1{:}n$, and generates a corresponding sequence of contextualized representations, $h1{:}n$.
2. A context vector, c, which is a function of $h1{:}n$, and conveys the essence of the input to the decoder.
3. A decoder, which accepts c as input and generates an arbitrary length sequence of hidden states $h1{:}m$, from which a corresponding sequence of output states $y1{:}m$, can be obtained

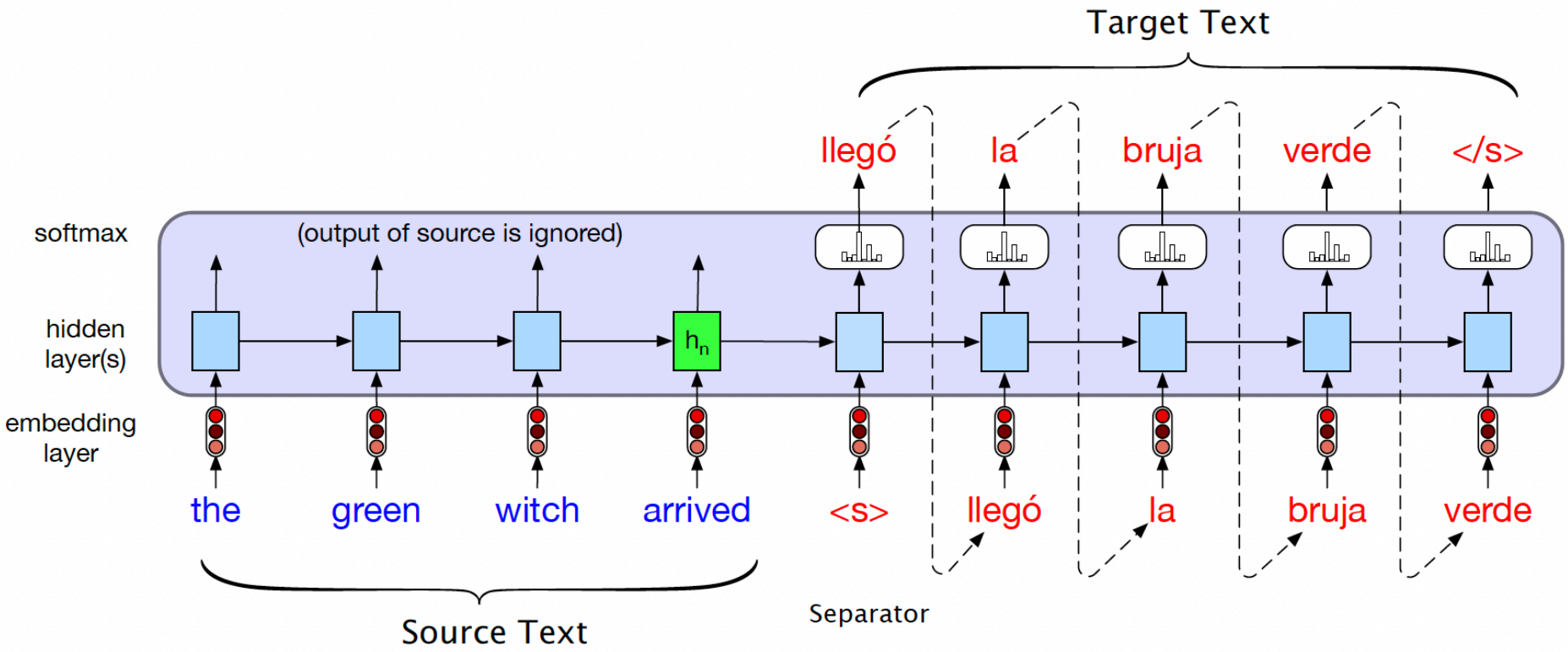
Encoder-decoder



Encoder-decoder

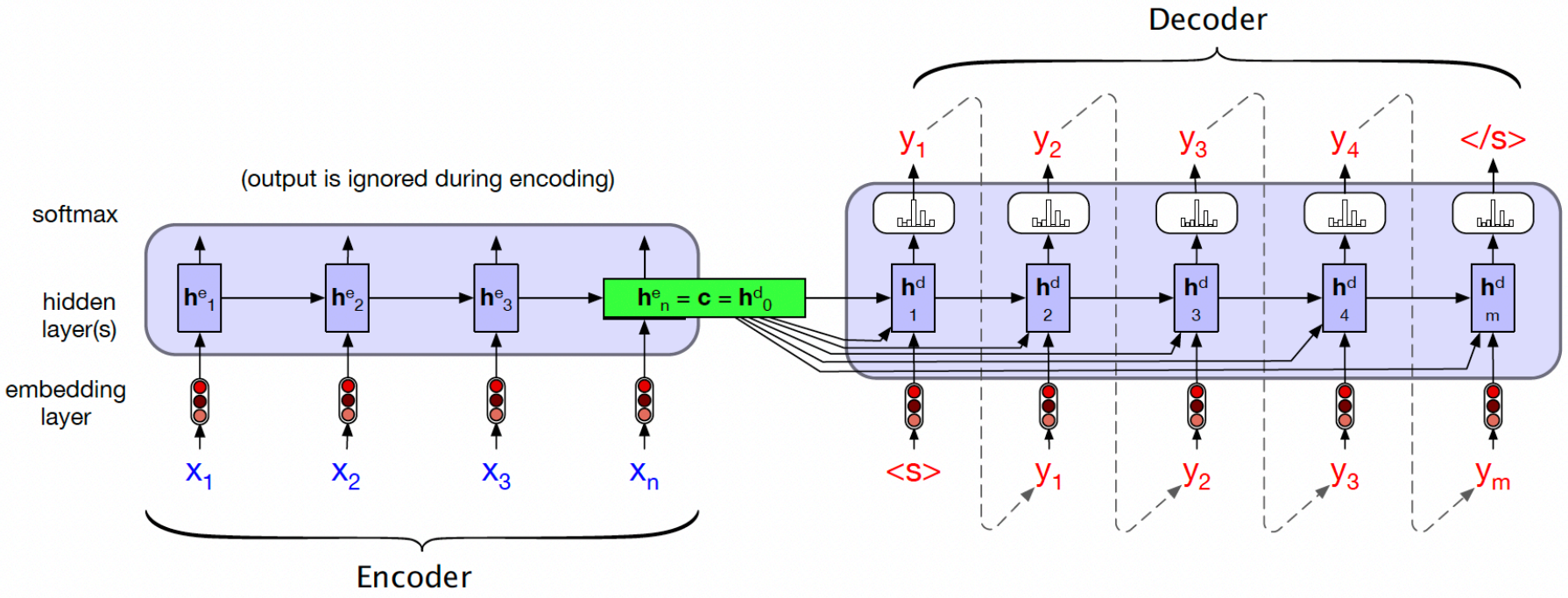


Encoder-decoder simplified



Encoder-decoder showing context

$$\mathbf{h}_t^d = g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c})$$



Encoder-decoder equations

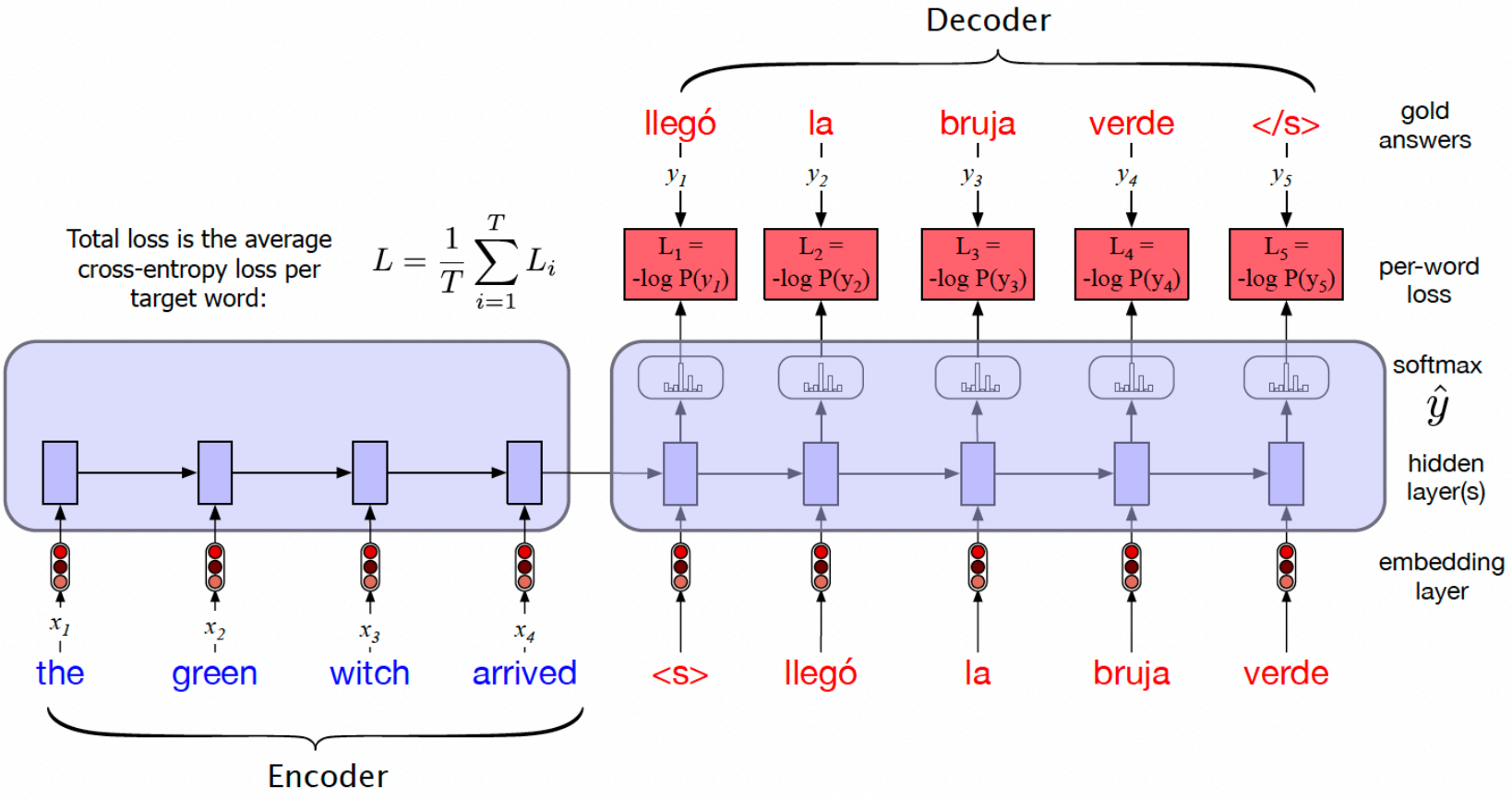
$$
\begin{aligned}
\mathbf{c} &= \mathbf{h}_n^e \\
\mathbf{h}_0^d &= \mathbf{c} \\
\mathbf{h}_t^d &= g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c}) \\
\hat{\mathbf{y}}_t &= \text{softmax}(\mathbf{h}_t^d)
\end{aligned}
$$

g is a stand-in for some flavor of RNN

$\hat{y}_{t-1}$ is the embedding for the output sampled from the softmax at the previous step

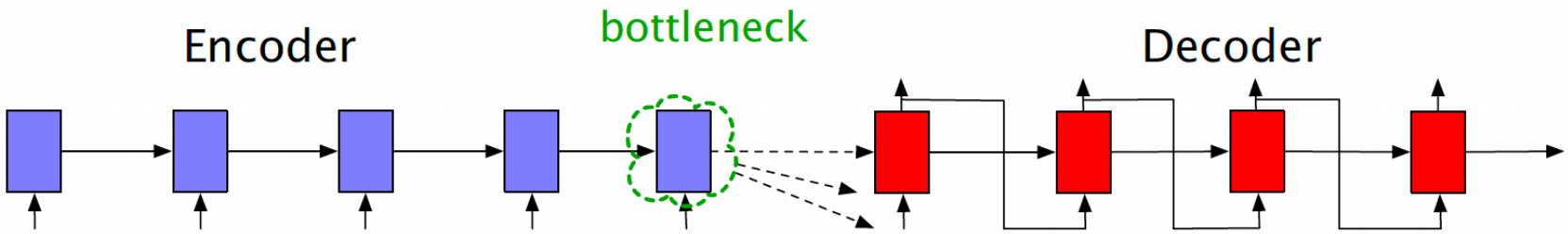
$\hat{y}_t$ is a vector of probabilities over the vocabulary, representing the probability of each word occurring at time *t*. To generate text, we sample from this distribution $\hat{y}_t$.

Training the encoder-decoder with teacher forcing



Problem with passing context c only from end

Requiring the context *c* to be only the encoder's final hidden state forces all the information from the entire source sentence to pass through this representational bottleneck.



Solution: attention

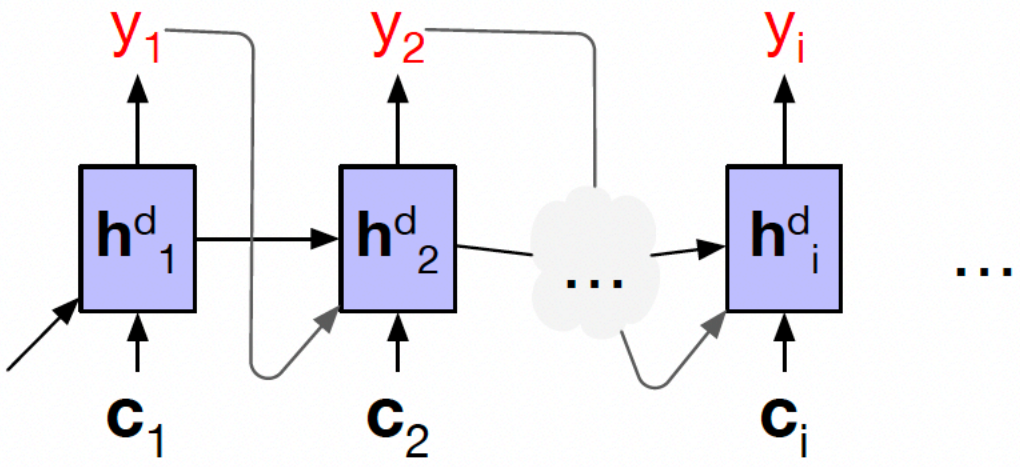
instead of being taken from the last hidden state, the context it's a weighted average of all the hidden states of the encoder.

this weighted average is also informed by part of the decoder state as well, the state of the decoder right before the current token *i*.

$$\mathbf{c} = f(\mathbf{h}_1^e \ldots \mathbf{h}_n^e, \mathbf{h}_{i-1}^d)$$

Attention

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i)$$



How to compute c?

We'll create a score that tells us how much to focus on each encoder state, how *relevant* each encoder state is to the decoder state:

$$\text{score}(\mathbf{h}^d_{i-1}, \mathbf{h}^e_j) = \mathbf{h}^d_{i-1} \cdot \mathbf{h}^e_j$$

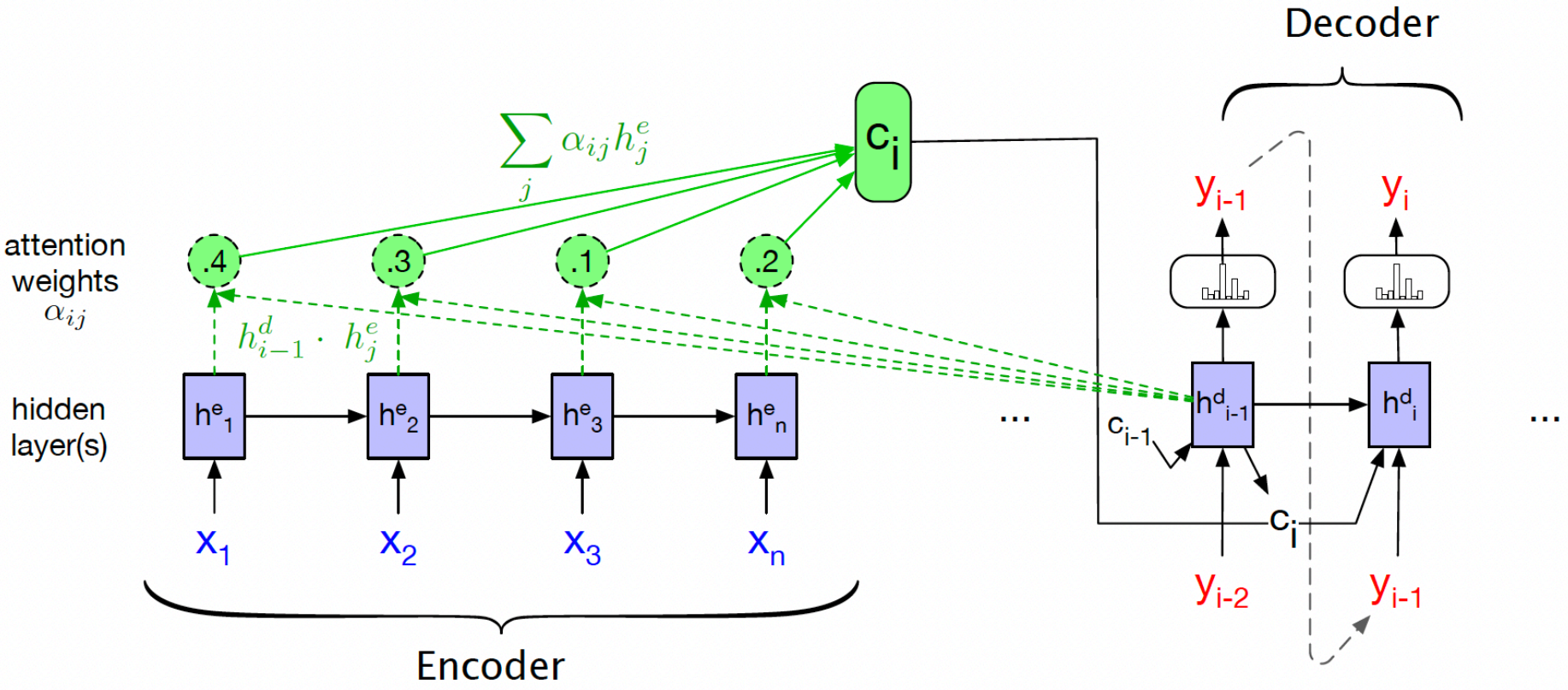
We'll normalize them with a softmax to create weights α_{ij}, that tell us the relevance of encoder hidden state *j* to hidden decoder state, h^d_{i-1}

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{h}^d_{i-1}, \mathbf{h}^e_j))$$

And then use this to help create a weighted average:

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}^e_j$$

Encoder-decoder with attention, focusing on the computation of c



More sophisticated scoring functions for attention models

$$\text{score}(\mathbf{h}^d_{i-1}, \mathbf{h}^e_j) = \mathbf{h}^d_{t-1}\mathbf{W}_s\mathbf{h}^e_j$$

Transformer